



Faculty: BSC(Hons) Computer System Engineering

Name: Sadikshya Acharya

Student ID: 239679765

MODULE CODE: CET138

MODULE TITLE: Full Stack Development

MODULE LEADER: David Grey

SUBMISSION DATE AND TIME: Fri, 19 Jun 2026

ASSIGNMENT: 1

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

• I have used artificial intelligence tools to generate an idea for my assignment:	YES
• I have used artificial intelligence tools to write my assignment for me:	NO
• I have used artificial intelligence tools to brainstorm ideas for my assignment:	NO
• I have used artificial intelligence tools to correct my original assignment:	NO

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.
- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

Live demo Link: https://rihanashrestha567-star.github.io/Demos/javascript_demo.html

What is JavaScript

JavaScript is a language that helps make web pages more interesting. It lets developers create web pages that can do lots of things like move things around on the page and respond to what users do.

It lets developers change the way the page looks handle what users do send requests to servers, store data on the users computer and create logic that responds to what users do right away.

Modern JavaScript has lots of features like special functions, template literals and the ability to break things down into smaller parts. These features make JavaScript more powerful and easier to use so developers can create cool single-page applications.

JavaScript is really important for making web pages that users can interact with. It powers everything from things like checking if a form is filled out right to complex things, like games and interactive pictures.

Why This Topic Matters in Modern Web Development

JavaScript is the language that can run directly in web browsers so it is essential for creating the parts of web pages that users interact with. Without JavaScript web pages would be really boring. Would not be able to do much.

Modern web applications need JavaScript to create experiences that feel like apps. Developers who are good at JavaScript are in demand. To be a developer you need to understand how to change the way the page looks handle what users do send requests to servers and manage the state of the application.

How NepalRide Demonstrates JavaScript

NepalRide shows off JavaScript skills with lots of features. The navigation uses JavaScript to switch between sections without reloading the page. The login system has animated cars that move when you type and the password field has a cool headlight effect.

The booking system uses JavaScript to process form data and create ticket modals with random booking IDs. The quick-rent buttons fill out the booking form with the selected vehicle.

The impressive thing is the Rock-Paper-Scissors game, which is a fully functional game with themed choices a live battle arena and score tracking.

Code Example 1. Core Implementation

The animated login cars use JavaScript to update the page in time.

```
```Javascript

loginUsernameInput.addEventListener('input' (e) => {

length = e.target.value.length; const progressPct =

Math.min((length / 20) * 85, 85);

usernameCar.style.left = . %';

UsernameCar.classList.add('driving');

});

```
```

Code Example 2. Interactive Feature

The Rock-Paper-Scissors game logic uses JavaScript to determine the winner and update the scores. ```Javascript function playRPS(playerChoice) { choices = ['rock' 'paper' 'scissors']; const cpuChoice = choices[Math.floor(Math.random()*3)]; document.getElementById('playerFighter').innerHTML = choiceSvgs[playerChoice]; if (playerChoice === cpuChoice) { /* tie */ } if (/* win condition */) { playerScore++; } }

Visual Demonstrations & Screenshots from NepalRide

The JavaScript implementation in NepalRide is really visible in the animated login cars, the working booking system and the Rock-Paper-Scissors game. These features show how

JavaScript can be used to change the way the page looks handle what users do and create logic.

The RPS game has a battle arena with animated fighters live score tracking and themed choices which shows how JavaScript can be used to create engaging self-contained applications.

Figure 1: Animated Login System

The cars move along the road tracks as you type. The character counters update in time. The password toggle reveals a beam animation. This demonstrates how JavaScript can be used to update the page in time and handle what users do.

Figure 2: Rock-Paper-Scissors Arcade Game

The game is fully functional with themed choices, a live battle arena and score persistence. This shows how JavaScript can be used to create applications.

Key Features & Technical Implementation Details

NepalRide uses JavaScript patterns, including event listeners, dynamic content updates, state management, form processing and game logic. The RPS game demonstrates application development with proper separation of concerns.

All the JavaScript code is vanilla, which means it does not use any frameworks and is organized into functions with clear responsibilities.

Implemented Features in NepalRide:

- * Single-page application navigation with smooth section switching

* Challenges Faced & Solutions Applied

- * Challenge: Making the typing cars feel responsive and natural. Solution: Used percentagebased positioning with CSS transitions and added a driving class only during active typing with a timeout removal.

- * Challenge: Creating a RPS game. Solution: Used themed SVG choices created a visual battle arena with fighter elements implemented game logic with clear win conditions and added score persistence.

* Challenge: Managing interactive demos without conflicts. Solution: Created demo panes with show/hide logic and isolated event handlers for each technology demonstration.

Personal Reflection & Learning Outcomes

JavaScript was the enjoyable part of building NepalRide. It was really cool to turn web pages into something users can interact with. The animated login cars and the RPS game were particularly fun to create.

I learned that good JavaScript is about more than making things work. It is about creating delightful micro-interactions that make users smile. The headlight effect on password reveal and the race animation after login success are details that make the experience feel special.

Building the RPS game from scratch gave me confidence that I can create self-contained applications with proper logic, state management and visual feedback.

Learning Outcomes Achieved:

* Demonstrated DOM manipulation and event handling

* Conclusion

* NepalRide demonstrates strong JavaScript skills through its interactive features, particularly the animated login system and the fully functional Rock-Paper-Scissors arcade game. These features show the ability to create app-like experiences using vanilla JavaScript. The clean code organization and attention to micro-interactions indicate readiness, for more complex frontend development challenges.